

## CSE 341 — Concepts of Programming Languages

**WorkoutLang — D3: Example Programs & Test Report (Part 1)**

Gebze Technical University | Spring 2026 | Due: 8 May 2026

**Part A — Valid Programs (Parse Successfully)**

The three programs below exercise every major construct of WorkoutLang as defined in D1: all control structures (if/else, while, repeat), the structured workout/day construct, function definition and call, all five types, and the domain-specific set and load statements. Each program was confirmed to parse successfully and produce a correct AST using:

```
./workoutlang --dump-ast tests/valid1.wl
```

**Program 1 — valid1.wl: Upper Body Workout with Conditional Routine****Constructs**

workout, day, set (domain-specific), routine, int variable, if/else, repeat, load

**Source Code**

```
// Program 1: Upper body workout with a routine
workout UpperBody {
  day Monday {
    set benchpress      reps 4 x 10 rest 90s
    set overhead_press  reps 3 x 8  rest 60s
    set tricep_dip      reps 3 x 12 rest 45s
  }
  day Wednesday {
    set pullup reps 4 x 8  rest 90s
    set row    reps 3 x 10 rest 60s
  }
}
routine myWeek {
  int totalSets = 5
  load UpperBody
  if totalSets > 3 {
    print "Heavy week"
  } else {
    print "Light week"
  }
  repeat 4 weeks {
    load UpperBody
  }
}
```

**AST Output**

```
Parsing successful.
```

**Program**

```
WorkoutDecl(UpperBody)
  DayDecl(Monday)
    SetStmt(exercise=benchpress, sets=4, reps=10, rest=90s)
    SetStmt(exercise=overhead_press, sets=3, reps=8, rest=60s)
    SetStmt(exercise=tricep_dip, sets=3, reps=12, rest=45s)
  DayDecl(Wednesday)
    SetStmt(exercise=pullup, sets=4, reps=8, rest=90s)
    SetStmt(exercise=row, sets=3, reps=10, rest=60s)
RoutineDecl(myWeek)
  VarDecl(int totalSets)
    IntLit(5)
  Load(UpperBody)
```

```

If
  Condition:
    BinOp(>)
      Ident(totalSets)
      IntLit(3)
  Then:
    Print
      StringLit("Heavy week")
  Else:
    Print
      StringLit("Light week")
Repeat(4 weeks)
  Load(UpperBody)

```

## Program 2 — valid2.wl: Function Declaration and While Loop

### Constructs

func declaration with params, return, workout, while loop, function call, var assignment

### Source Code

```

// Program 2: Functions, while loop, expressions
func totalVolume(s: int, r: int) : int {
  return s * r
}
workout LegDay {
  day Friday {
    set squat      reps 5 x 5  rest 3min
    set legpress   reps 4 x 12 rest 90s
    set lunge      reps 3 x 10 rest 60s
  }
}
routine strengthPlan {
  int week = 1
  while week <= 12 {
    load LegDay
    int vol = totalVolume(5, 5)
    if vol > 20 {
      print "High volume"
    }
    week = week + 1
  }
}

```

### AST Output (excerpt — key nodes)

Parsing successful.

```

Program
  FuncDecl(totalVolume) -> int
    Param(s: int)
    Param(r: int)
    Return
      BinOp(*)
        Ident(s)
        Ident(r)
  WorkoutDecl(LegDay)
    DayDecl(Friday)
      SetStmt(exercise=squat, sets=5, reps=5, rest=3min)
      SetStmt(exercise=legpress, sets=4, reps=12, rest=90s)
      SetStmt(exercise=lunge, sets=3, reps=10, rest=60s)
  RoutineDecl(strengthPlan)

```

```

VarDecl(int week) -> IntLit(1)
While
  Condition: BinOp(<=) Ident(week) IntLit(12)
  Body:
    Load(LegDay)
    VarDecl(int vol) -> Call(totalVolume) [IntLit(5), IntLit(5)]
    If Condition: BinOp(>) Ident(vol) IntLit(20)
      Then: Print StringLit("High volume")
    Assign(week) -> BinOp(+) Ident(week) IntLit(1)

```

### Program 3 — valid3.wl: Bool Type, Logical Operators, Nested Repeat

#### Constructs

bool type, && and ! operators, nested if inside repeat, func returning bool, all DURATION suffixes

#### Source Code

```

// Program 3: All types, nested if, bool expressions
func isHeavy(s: int, r: int) : bool {
  return s * r > 30
}
workout FullBody {
  day Monday {
    set deadlift      reps 3 x 5  rest 3min
    set benchpress    reps 4 x 8  rest 2min
  }
  day Thursday {
    set squat         reps 4 x 6  rest 2min
    set overhead_press reps 3 x 10 rest 90s
  }
}
routine periodization {
  repeat 8 weeks {
    load FullBody
    bool heavy = isHeavy(4, 8)
    if heavy && true {
      print "Deload recommended"
    }
    if !heavy {
      print "Add weight next session"
    }
  }
}

```

#### AST Output (excerpt — key nodes)

Parsing successful.

```

Program
  FuncDecl(isHeavy) -> bool
    Param(s: int) Param(r: int)
    Return BinOp(>) [ BinOp(*) Ident(s) Ident(r) ] IntLit(30)
  WorkoutDecl(FullBody) [ Monday + Thursday days ]
  RoutineDecl(periodization)
    Repeat(8 weeks)
      Load(FullBody)
      VarDecl(bool heavy) -> Call(isHeavy) [IntLit(4), IntLit(8)]
      If Condition: BinOp(&&) Ident(heavy) BoolLit(true)
        Then: Print StringLit("Deload recommended")
      If Condition: UnOp(!) Ident(heavy)
        Then: Print StringLit("Add weight next session")

```

## Part B — Malformed Programs (Parser Error Messages)

The five programs below each contain a deliberate syntax error. For each, the source code, the exact error output from the parser, and the reported line number are shown.

### Error Summary Table

| #  | Source (malformed)                                                      | Error Message                               | Line |
|----|-------------------------------------------------------------------------|---------------------------------------------|------|
| E1 | workout Chest { day Monday<br>{ set bench reps 3 x 10<br>rest 60s [EOF] | expected '}' at end of day block            | 6    |
| E2 | set pullup reps 4 10 rest<br>90s                                        | expected 'x' between sets and reps          | 4    |
| E3 | int vol @ 25                                                            | Unknown char '@'                            | 10   |
| E4 | set benchpress reps 4 x 10<br>[no rest]                                 | expected 'rest' after rep count             | 5    |
| E5 | int x = 5 [top level]                                                   | expected 'workout', 'routine', or<br>'func' | 2    |

### Error 1 — error1.wl: Missing Closing Brace

#### Source

```
workout Chest {
  day Monday {
    set benchpress reps 3 x 10 rest 60s
    // closing braces missing
```

#### Parser Output

```
[line 6] Parse error at 'EOF': expected '}' at end of day block
[line 6] Parse error at 'EOF': expected '}' at end of workout
Parsing failed — see errors above.
```

### Error 2 — error2.wl: Missing 'x' Between Sets and Reps

#### Source

```
workout Back {
  day Tuesday {
    set pullup reps 4 10 rest 90s
  }
}
```

#### Parser Output

```
[line 4] Parse error at '10': expected 'x' between sets and reps
[line 4] Parse error at 'rest': expected 'INT_LIT' rep count
[line 4] Parse error at '90s': expected 'rest' after rep count
[line 5] Parse error at '}': expected 'DURATION_LIT' rest duration
[line 7] Parse error at 'EOF': expected '}' at end of workout
Parsing failed — see errors above.
```

### Error 3 — error3.wl: Unknown Character '@'

#### Source

```
routine plan {
  load Legs
  int vol @ 25
}
```

#### Parser Output

```
[line 10] Parse error at 'Unknown char '@': Unknown char '@'
[line 10] Parse error at '25': expected '=' after variable name
```

Parsing failed — see errors above.

## Error 4 — error4.wl: Missing Rest Duration in Set Statement

### Source

```
workout Push {  
  day Monday {  
    set benchpress reps 4 x 10  
  }  
}
```

### Parser Output

```
[line 5] Parse error at '}': expected 'rest' after rep count  
[line 6] Parse error at '}': expected 'DURATION_LIT' rest duration  
[line 7] Parse error at 'EOF': expected '}' at end of day block  
[line 7] Parse error at 'EOF': expected '}' at end of workout  
Parsing failed — see errors above.
```

## Error 5 — error5.wl: Statement at Top Level

### Source

```
int x = 5  
print x
```

### Parser Output

```
[line 2] Parse error at 'int': expected 'workout', 'routine', or 'func'  
[line 2] Parse error at 'x': expected 'workout', 'routine', or 'func'  
[line 2] Parse error at '=': expected 'workout', 'routine', or 'func'  
[line 2] Parse error at '5': expected 'workout', 'routine', or 'func'  
[line 3] Parse error at 'print': expected 'workout', 'routine', or 'func'  
Parsing failed — see errors above.
```